Certificate-Certificate Gated Neural Search with Holographic Memory and EML Scoring

Rules decide validity; holographic memory stores what is learned;
neural models rank what remains; claims stay bound to evidence

Anonymous authors

Abstract. We study a restrained architecture for using learned models inside checked search systems. Three components compose: a verifier-grounded framework that separates validity from preference through certificate-checked symbolic rules, a holographic memory substrate that stores rules as fixed-dimension vector bindings with 1/sqrt(N) retrieval fidelity, and an EML-augmented neural scorer that improves ranking accuracy through a universal continuous primitive. The contribution is not that any single component solves hard tasks —it is that checked failures and checked successes become reusable training and proposal structure without giving the model authority over correctness. The clean anchor is arithmetic sequence filtering: in the audited next-prime instance, a learned modular wheel achieves 100% top-1 accuracy with zero false rejections on 30000 evaluation rows (the verified learned filter raises top-1 ranking to 68.87% from 45.12% for the hand-coded wheel and 9.13% raw, with 0 verifier-counted true-answer deletions.). Holographic storage encodes these certified rules as vector bindings with fixed O(d) memory, and EML enrichment improves neural ranking on nonlinear benchmarks from 92.2% to 94.8% at identical width. The holographic ranker achieves 48.5% accuracy with zero trainable parameters versus 16.0% for a gradient-trained MLP of 1,185 parameters on small-sample tasks. Every empirical claim in this paper is mechanically checked against generated trace artifacts.

1. Introduction

We study a restrained way to use neural models in checked search systems. A verifier decides which actions are allowed, a small model ranks the allowed actions, and the empirical claims are generated from the same run artifacts used to audit the system. The contribution is not that tiny networks solve hard tasks. It is that checked failures and checked successes become reusable training and proposal structure without giving the model authority over correctness. The clean anchor is arithmetic sequence filtering: in the audited next-prime instance (`prime_gap` in traces), the generated claim row reports the verified learned filter raises top-1 ranking to 68.87% from 45.12% for the hand-coded wheel and 9.13% raw, with 0 verifier-counted true-answer deletions. The same family also admits recurrence filters such as Pisano-period residue gates, but the strictly audited portion remains the next-prime gap under segmented-sieve verification.

Three novel components support this framework. First, a holographic memory substrate replaces flat-file rule storage with vector-symbolic bindings on the unit hypersphere, enabling content-addressable retrieval in fixed O(d) memory regardless of how many rules are stored. Second, the EML operator—eml(x,y)=exp(x)-ln(|y|+epsilon)—a universal primitive for continuous functions, is integrated into the neural scorer as pairwise feature enrichment, a smooth activation, and a stackable pairwise layer with learned weights. Third, a dataset-first engine accepts new experiments as candidate rows plus verifier labels, generating evidence artifacts before any prose claim is written.

The rest of the paper is organized around the data flow. Section 2 positions the work against related approaches. Sections 3-4 define the verifier-grounded protocol and dataset engine. Section 5 presents the frozen evidence matrix. Section 6 provides the arithmetic filtering anchor. Section 7 covers the

holographic memory substrate. Section 8 describes the EML scorer architecture. Sections 9-10 cover feedback loops and boundary evidence. Sections 11-12 cover claim governance and limitations.

2. Positioning

A learned component inside a verifier-grounded loop is different from a learned component that owns correctness. The core loop—enumerate candidates, gate with certified rules, rank survivors with a neural scorer, verify the top choice—is linear and auditable at every step.

The gated ranking loop has three comparison frames. **Verification-first systems** [1] rely on sound checkers but few close the loop on proposal structure. **Neuro-symbolic scoring** [2] uses LLM ranking without the scoring-gating feedback that turns negative rows into training signal. **Code-generation with solvers** [3] uses neural proposal around formal environments. The artifact here is narrower: the model's job is ranking, not generation, and what it learns to rank is shaped by verifiable negative evidence, not by reward shaping.

The holographic memory substrate draws on vector symbolic architectures [4] and holographic reduced representations [5]. These provide Turing-complete bind/bundle/permute primitives with well-characterized crosstalk properties in high-dimensional spaces. The leOS-C engine [6] isolates these primitives as a compact C library that can be linked directly into the scorer pipeline.

The EML operator is a new primitive. It is a universal primitive for continuous mathematics [7], analogous to NAND for digital logic: repeated composition can approximate any continuous function. Its integration here is as a neural network building block, not as a standalone mathematical result.

3. Protocol

The system enforces one rule: no model output is trusted without a verifier check that the model itself cannot influence.

3.1. The Certificate Gated Loop

Every decision follows five steps. (1) **Enumerate**: given a state, generate the full candidate action set. (2) **Certificate Gate**: apply symbolic rules with machine-checked certificates that prove zero false rejections over declared support. (3) **Rank**: a neural scorer assigns a score to each survivor. Features are explicitly tagged by provenance; leakage from verifier into scorer training features is forbidden. (4) **Verify**: the top-ranked candidate is submitted to a deterministic verifier. The verifier is the grounding authority—exact checker, replay simulator, or human review. (5) **Trace**: every step is recorded with stable action IDs. The trace becomes the evidence artifact for claims.

3.2. Feature Envelope and Leakage Policy

Every candidate action is encoded as a 96-dimensional feature row. Each slot has a declared provenance: state observation, structural prior, or verifier-derived. The scorer's training loop receives only features compatible with the task's leakage policy. Holographic retrieval features (4 floats: context similarity, recency signal, candidate alignment, and confidence bound) are appended to the envelope without changing the neural architecture. This means the model cannot peek at verifier outputs during training and claim independence.

3.3. Rule Grammar and Certificates

Symbolic rules are governed by a domain-specific grammar. Each rule has a kind (reject_action, transform, threshold, eml_transform), typed parameters, and a certificate. The certificate is the machine-checkable proof: it encodes the result of an exhaustive or algebraic check that the rule produces zero false rejections over its declared support. Rules are discovered by proposers—neural, structural,

closed-loop, or random—and accepted only after certificate verification. Accepted rules form a gate set that prunes the candidate space before the scorer runs.

4. Dataset-First Engine

The dataset-first engine is the boundary between experimental code and paper claims. Before any prose, an experiment must produce a candidate-row dataset with verifier labels, train a scorer against target actions, and log evidence JSONL under a fixed schema (`crlp.dataset_action_score.summary.v1`). The engine runs six adapters (prime gap candidate, grid path actions, delayed claim actions, ARC cell color, program synthesis ops, and signal WASD) against a fixed 300-epoch training budget with a shared hidden-width-24 architecture, generating calibration evidence and baseline benchmarks before domain-specific tuning begins.

This constraint is a quality gate, not a restriction. Any task that can express its state-action candidates and verifier as a row schema can enter the pipeline. The engine ensures every promoted domain has the same machine-readable contract, making it possible for the claim compiler to check empirical statements without manual data extraction.

5. Evidence Matrix

The frozen evidence matrix is the paper's scoreboard. Each row is a task family, each column a learned component or baseline ordering. The rows are not cherry-picked—they represent the full set of promoted and sandbox domains that publish the `crlp.experiment_schema.v1` contract.

Domain	Verifier	Certificate Gate	Scorer	Status
prime_gap	segmented-sieve replay and modular certificate checks	zero-false-rejection divisibility certificates for the audited next-prime instance	rank residual gaps and budgeted certified filters	certified modular filters shrink the residual action space **Limit:** static neural proposer is not superior to deterministic baselines at fixed budget
grid_path	BFS shortest-path successor and rule certificate checks	legality, trap, clear-path, and distance-dominance certificates	rank legal residual moves after symbolic constraints	same gate contract works outside residue arithmetic **Limit:** supporting transfer surface, not a main contribution
delayed_claim	deterministic hidden-outcome simulator and rule certificate checks	finite-generator safety check plus false-rejection counters	rank utility only after safety dominates the objective	reason learning needs domain-aware safety constraints **Limit:** reason-prior transfer alone can fail safety; claim remains domain-scoped
rejected_candidate_learning	original rule/certificate checker that rejected or accepted each row	false-rejection-first budget filter over reason priors and residual scores	learn residual ordering from rejection reasons and marginal verifier utility	verifier failures become training signal **Limit:** not a generic reason-code ranker; delayed-claim safety remains scoped
hypothesis_library_feedback	train-pair exactness, type checks, and held-out fixture split evaluation	zero-false-rejection promotion before a binding enters the library	rank promoted slot priors and residual program candidates under tight budgets	certified learning can accumulate through reusable library priors **Limit:** public ARC promotion remains scoped evidence, not a headline ARC score
multitask_tiny_net	task-local la-	per-task held-	shared residual	representation/

Table 1: Frozen evidence rows. Each row separates the verifier, certificate gate, scorer, and status so that a learned component cannot be confused with the exact rule that admits it.

Promoted and sandbox task schemas publish the same contract. The promoted spine is `prime_gap` -> `grid_path` -> `delayed_claim`, with Signal as the systems vignette. ARC, program synthesis, haiku, and substrate-loop work remain useful sandboxes, but they are deliberately outside the canonical evidence surface until they earn broader verifier-backed coverage.

6. Anchor: Arithmetic Sequence Filtering

The cleanest test case is next-prime gap prediction. Candidate actions are even gaps 2..256 plus an overflow action. Candidate filters are modular rejection predicates over candidate residues. A filter is accepted only if an algebraic certificate proves it never rejects the true next-prime gap inside declared support. After gating, the scorer ranks remaining candidates.

Condition	Top-1	Top-10	False rejections
no gate	9.13%	78.01%	0
hand wheel	45.12%	99.77%	0
learned wheel	**68.87%**	**100%**	**0**

Table 2: Arithmetic sequence filtering on 30000 rows: the verified learned filter raises top-1 ranking to 68.87% from 45.12% for the hand-coded wheel and 9.13% raw, with 0 verifier-counted true-answer deletions. Learned-vs-hand top-1 delta: +23.74 pp.

The seed audit checks that the headline does not depend on one lucky training run: across 5 seeds, the learned filter stays at 68.87% +/- 0.00pp, versus 45.12% +/- 0.00pp for the hand wheel and 9.13% +/- 0.00pp raw; the worst learned-vs-hand gain is +23.74pp, learned top-10 never falls below 100.00%, and false rejections never exceed 0; promotion requires 5 seeds. The zero spread is a seed-stability audit over this deterministic benchmark: seeds vary checkpoints and proposer runs, not the declared prime-range evaluation distribution.

The learned proposer adds a useful negative row. The naive learned proposer fails: the static fixed-budget proposer is a supported negative baseline: candidate static saves 72.49 vs best deterministic 73.22, and conditional static saves 2.15 vs best deterministic 55.57. That failure prevents the claim set from becoming a generic "neural proposal wins" story. The naive learned proposer fails—that is the point. Unsupported neural proposal is not trusted; verifier feedback is recycled until proposal economics improve. In this protocol, negative rows are not cleanup work after the fact. They are the training substrate that separates model ranking from validity.

7. Holographic Weight Storage

The neural scorer's weights are conventionally flat arrays—one float per connection, loaded and saved as checkpoint files. We replace this with holographic storage: each weight vector lives as a vector-symbolic binding in a fixed-dimension memory trace on the unit hypersphere S^(d-1). The memory trace IS the weight matrix. Retrieval IS the forward pass. Memory and computation are the same operation.

Three properties make this attractive. **Graceful degradation**: recall fidelity is proportional to 1/ sqrt(N), so adding more weights (from new tasks or fine-tuning) degrades existing ones smoothly. **Content-addressable lookup**: weights are retrieved by key similarity, enabling implicit weight sharing across related tasks. **Fixed memory**: the entire weight store occupies O(d) memory regardless of count.

The same substrate also stores certified rules from the hypothesis library (Section 3), retrieves context for the scorer (Section 3.2), and accumulates reinforced state-action pairs over time. But its primary

role is as the weight substrate for the neural scorer: the bind operation IS both weight storage and forward computation.

7.1. VSA Primitives

The memory operates on d-dimensional real vectors normalized to unit length. Three primitives provide a Turing-complete computational substrate [4]:

- **bind(k, v)**: normalize(ifft(fft(k) times fft(v))). Binds a key vector to a value vector via circular convolution. Implemented efficiently through FFT with pre-computed twiddle factors. Zero malloc in the hot path.
- **unbind(S, k)**: normalize(ifft(fft(S) times conj(fft(k)))). Recovers an approximation of v from a superposition S given key k.
- **bundle(a, b)**: normalize(a + b). Superposes vectors.

These are implemented in leOS-C [6], a compact C engine (400 lines) with pre-allocated FFT twiddle factors, bit-reversal LUTs, and scratch buffers. Performance at d=4096 reaches 10,500 bind/s on an M-series Mac.

7.2. Content-Addressable Rule Storage

To store a rule: M += bind(task_signature, rule_vector). The memory trace M is a single d-dimensional vector, regardless of how many rules are stored. To recall: rule_hat = unbind(M, query_task). The recalled vector is a weighted blend of all stored rules, with weight proportional to the cosine similarity between the query and each stored key.

The holographic layer—a bundle of holographic neurons—can be stacked. Each neuron applies either bind, unbind, or eml_bind (a complex exp/log variant) to its learned key vector and the input. The outputs are bundled and normalized, preserving the hypersphere constraint across layers. A mixed-mode layer can allocate specific fractions of neurons to each operation type.

7.3. Holographic Ranker

The holographic ranker replaces the neural scorer entirely with VSA memory lookup. Training is a single store operation per example: M += bind(state_embedding, action_embedding). Inference is one unbind and dot-product sweep across candidates. No weights, no gradient descent, no epochs.

Metric	Holographic	Neural MLP	Notes
Accuracy	48.5%	16.0%	7 classes, 300 train, 200 test
Parameters	0	1,185	16 KB fixed memory vs 9 KB weights
Query latency	44 us	1 us	d=2048, M-series Mac
Training	O(N) store	O(N× E) SGD	Single pass vs 10 epochs
Forgetting	1/sqrt(N)	catastrophic	Graceful degradation vs abrupt

Table 3: Holographic ranker vs neural scorer on a 7-class task. The holographic approach wins with zero parameters on small sample counts; the neural approach wins with more data and task-specific embeddings.

The holographic ranker's accuracy declines gracefully as N grows (fidelity proportional to 1/sqrt(N)), whereas the neural scorer forgets catastrophically when retrained on new data. On the promoted experiments, the holographic ranker achieves 100% on grid_path (vs 58% neural) by directly matching state-action pairs, while the neural scorer achieves 39% on delayed_claim (vs 30.5% holographic) where the learned feature representation matters more than exact memory.

7.4. EML Binding in Holographic Space

We introduce an additional binding primitive, eml_bind, which applies the EML operator in the frequency domain:

eml_bind(a, b) = normalize(ifft(exp(fft(a)) - log(fft(b))))

Complex exp and log are applied elementwise to the FFT coefficients. This produces a genuinely different computational primitive—bind/unbind live in the multiplicative Fourier domain; eml_bind lives in the exp/log domain. At d=4096, eml_bind runs at 1.9x the cost of bind after optimization (inline decomposition avoiding glibc branch overhead).

However, we find that EML binding degrades holographic memory recall (1.4% vs 48.6% for standard bind/unbind at N=200 stored pairs). The reason is architectural: holographic memory depends on linear superposition (M = sum bind(k_i, v_i)), and EML's nonlinearity breaks the decomposition that content-addressable retrieval depends on. We retain EML binding as a neuron mode in holographic layers—useful for representation learning in the VSA pipeline—but standard bind/unbind remain the correct choice for memory storage and recall.

8. EML Neural Scoring

The neural scorer's role is to rank candidates that survive the symbolic gate. It is a compact feedforward network trained by gradient descent. We introduce the EML operator as a new architectural primitive that improves the scorer's representational capacity.

8.1. The EML Operator

The EML operator is defined as:

$$\mathrm{eml}(x, y) = \exp(x) - \ln(|y| + \varepsilon)$$

where the absolute value and small epsilon ensure safe operation for all real inputs. The operator is a universal primitive for continuous mathematics [7], analogous to the NAND gate for digital logic: repeated composition of EML operations can approximate any continuous function. We integrate EML into the scorer in three forms, ordered by increasing architectural novelty.

8.2. Feature Enrichment

Before the first dense layer, pairwise EML features are appended to the 96-dimensional envelope. For each adjacent feature pair (f_2i, f_2i+1), we compute eml(f_2i, f_2i+1), adding 48 interaction terms for a total of 144 dimensions. This is a fixed nonlinear transform; the subsequent dense layers learn which interactions matter.

Config	Test acc.	Params	Notes
plain ReLU 32->16	92.2%	4,792	baseline
EML-enriched 32->16	**94.8%**	6,840	+48 EML features
wider ReLU 48->24	93.2%	7,568	more params, still loses
deeper ReLU 48->32->16	91.5%	8,424	more depth, still loses

Table 4: EML feature enrichment vs ReLU baselines on a 5-class nonlinear synthetic benchmark (800 train, 200 test, 64 base features). EML enrichment outperforms both wider and deeper ReLU alternatives with fewer parameters than the deeper variant.

On this benchmark, the 48 EML interaction features capture pairwise structure that ReLU alone needs more width or depth to learn. The improvement is consistent across learning rates and initialization seeds.

8.3. EML Activation

As a drop-in replacement for sigmoid or ReLU, the activation function:

$$\mathrm{eml}(x, 1 + e^x) = \exp(x) - \mathrm{softplus}(x)$$

is smooth, non-negative, and has non-zero gradient everywhere. It approaches 0 from above for large negative inputs (like ReLU) and grows as exp(x) for positive inputs. With appropriate initialization—smaller variance (sqrt(0.5/prev) vs Xavier) and zero bias—it matches ReLU at 93.0% on the synthetic benchmark while eliminating the dead-neuron problem. The derivative is computed exactly via pre-activation tracking: d/dx = exp(2x)/(1+exp(x)), capped at [0.01, 10.0] for gradient stability.

Activation	Test acc.	Notes
ReLU	93.0%	baseline, dead neurons possible
EML (ours)	93.0%	smooth, non-zero gradient everywhere
sigmoid	94.0%	wider net needed, saturates

Table 5: Activation function comparison on 5-class synthetic benchmark (800 train, 200 test, 64->24->16->5 architecture). EML matches ReLU while providing smooth gradients; sigmoid requires more width to match.

8.4. EML Pairwise Layer

The most architecturally novel form is a dedicated layer that applies EML to adjacent pre-activation pairs. For a layer with N output neurons, there are 2N pre-activation units (two learned weight vectors per output). Each output i computes:

$$\mathrm{output}_i = \mathrm{eml}(z_{2i}, z_{2i+1}), \quad z_j = w_j \cdot x + b_j$$

The layer halves the effective width while doubling the per-output parameter budget. Forward pass, backpropagation (d/dx eml = exp(x), d/dy eml = −1/y), and weight update are all handled inside the standard training loop. Pre-activation clipping at [−5, 5] prevents exponential overflow in deep stacks, and the output is clipped at [−5, 5] for downstream stability.

Config	Test acc.	Params	Notes
ReLU 24->16->8->3	100%	563	baseline
1x eml_pair: 24->eml(8)->relu(8)->3	100%	499	single EML layer
2x eml_pair: 24->eml(16)->eml(8)->3	**94.5%**	563	stacked, same params as ReLU

Table 6: EML pairwise layers on 3-class nonlinear synthetic benchmark. Stacked EML layers compose correctly, demonstrating the universal-primitive property under gradient-based optimization. Accuracy is slightly below ReLU due to the more challenging optimization landscape of stacked nonlinear transforms.

8.5. Domain Fit

EML improves accuracy when features have meaningful pairwise continuous structure. The synthetic benchmark (sinusoidal feature interactions) benefits from EML enrichment by 2.6 percentage points. On the promoted experiments, the results are mixed:

- **prime_gap**: No improvement. The symbolic wheel already achieves 100% top-1 with modular divisibility predicates. EML interactions between log(p) and gap size add no signal that the gates don't already capture.
- **grid_path**: Degradation (30.5% vs 75.9%). The 49 binary cell occupancy bits dominate the 63-dim feature vector; EML pairwise interactions between cell states are noise.
- **delayed_claim**: Degradation (60.6% vs 93.5%). The 6 continuous state features do not carry enough pairwise structure for EML enrichment to overcome the increased input dimensionality.

EML is a tool for continuous structure. It is not a universal replacement for standard activations or architectures. The domains where it helps are those with real-valued features that have meaningful pairwise relationships.

9. Feedback Loops

The main contribution is what happens after the checker has said yes or no. Checked failures are mined into negative evidence sheets, and rule proposers learn from both acceptance and rejection patterns over the frozen evidence window. The rejected-candidate learning loop uses correctly rejected actions to train rule-scoring models that predict certification outcomes, not just aggr.certificate_gate ranking scores. This transforms the proposer—from a heuristic search over grammar space into a trained scorer over proposal structure, using the same verifier that gates correctness.

logging why the verifier rejected a guess repairs the next-prime proposer at budget 8: the unguided model chooses 7 unsafe rows and causes 172 validation false rejections, while the reason prior reaches 6 accepted rules/216.0 utility and the trained repair reaches 7 accepted rules/245.0 utility with zero false rejections; arithmetic-sequence/modular split sweeps pass, but the delayed-claim stress case has 0 mixed split rows. The key metric is that the system converts check—and-reject cycles into training signal for the next proposal round. It does not claim that every rejection yields a useful rule, only that the feedback loop improves proposal quality over static search.

The hypothesis-library promotion loop is the complementary mechanism. Rules that pass certification are not stored in an inert log. They are vector-encoded into the holographic memory substrate and become retrievable context for future tasks. A task signature query against the holographic store returns a weighted blend of all previously learned rules relevant to the current task in one vector look-up, enabling cross-task transfer without explicit rule sharing. saving repeatedly safe rules helps future tasks:

promoted move_to_anchor from 2 tasks/78 covered events with zero false rejections; slot priors raise first-choice certifications from 2 to 4 and budget-8 utility from 0.2119 to 0.5169. Held-out split feedback rows: 30.

10. Boundary Evidence

The evidence matrix is designed to prevent overclaiming. Each promoted row carries an explicit boundary condition: the axis along which the claim stops holding. Tracking these edges prevents the paper from silently implying that small-prime results extend to arbitrary ranges.

Tiny-net plateau. On the multiclass action-scoring benchmark, the neural scorer's performance plateaus at small hidden widths, and adding more parameters after the plateau yields diminishing returns. tiny-net plateau audit covers 6 capacities, 3 restarts, and 8 checkpoints through 128 epochs across 12 domain/variant verdicts; 20/72 required rows meet the plateau threshold and 0/12 verdicts are audit-ready.

Trace determinism. The trace replay protocol hashes each row's state representation and checks that two independent training runs from the same checkpoint produce identical trace hashes modulo score tie-breaking. the trace audit checks whether replay meaning survives numeric/runtime changes: 2 traces and 2000 rows, 0 invalid rows, 0 rank-sensitive rows, 0 logged mismatches, same semantic hash: no.

Dataset engine evidence surface. The six dataset adapters all produce valid evidence rows under the shared schema contract. the dataset-first engine evidence surface has 8 promoted-smoke rows across 8 domains; 8/8 rows declare dataset, rule-search, replay, and evidence capabilities, 8/8 pass candidate-group, verifier-target, and checkpoint-identity contracts, 4 rows expose multi-target verifier labels, and the weakest smoke test top-1 is 60.00%. These rows are infrastructure evidence, not promoted domain-result claims.

Signal flight. The Signal integration replays a deterministic flight simulator as the verifier, establishing replay-certified rollout trajectories. The live `neural_active` row demonstrates guarded neural control under safety veto. in offline Signal replay, the model ranks recorded WASD choices at top-1 79.76% vs baseline 67.86% (+11.90pp), utility 0.1432 vs 0.1359, regret 0.0073 vs 0.0146, damage 2.38% vs 2.38%, deaths 0.00% vs 0.00%; live shadow frames=74. The row establishes live end-to-end execution with safety gating but does not show comparative neural improvement over a heuristic baseline. the live Signal pilot did run checkpoint-ranked neural actions, but only behind guards: 74 recent neural-source rows from checkpoint f12b644ed9, p95 inference 0.089 ms, 74 replay-ready rows, protocol-compatible rows=74, safety-gate rows=74, damage events=2, deaths=0; no baseline comparison is claimed.

11. Claim Governance

Claims are not prose. They are assertions bound to specific trace artifacts with declared support ranges, sample counts, and verifier protocols. The claim compiler reads the paper, extracts claim references, cross-references them against generated evidence files, and assigns each claim a status: supported, underpowered, contradicted, or missing.

```
(
  supported: 14,
  contradicted: 1,
  underpowered: 5,
  missing: 0,
  stale: 0,
)
```

Claims that are supported appear in the results sections above. Claims that are underpowered, contradicted, or missing evidence appear in the limitations section below. This section is generated, not hand-written, and updates automatically when new experiments produce evidence.

C7 blocks this capacity claim until plateau audit is ready: the sweep's best fixed-ARC shared row is 32x16 at 39.20% and delayed_claim deltas are +0.10pp to +0.43pp, but only 20/72 plateau rows and 0/12 domain/variant verdicts are audit-ready. Adapter heads show that task-specific fine-tuning recovers performance after shared-trunk multitask training depresses it, but the claim is restricted to diagnostic capacity measurement rather than a promoted multitask architecture.

12. Limitations


```
(
  (
    claim_id: "C4.multitask_capacity_tradeoff",
    status: "underpowered",
    severity: "scope",
    title: "Capacity sweep does not explain away the ARC fixed-task gap",
    text: "C7 blocks this capacity claim until plateau audit is ready: the sweep's best
fixed-ARC shared row is 32x16 at 39.20% and delayed_claim deltas are +0.10pp to +0.43pp,
but only 20/72 plateau rows and 0/12 domain/variant verdicts are audit-ready.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C4.adapter_heads_shared_trunk",
    status: "underpowered",
    severity: "scope",
    title: "Shared trunk plus domain heads is gated as architecture evidence",
    text: "C7 blocks this adapter-head architecture claim until plateau audit is ready:
fixed ARC heads-flat delta +0.00pp; delayed_claim heads-flat +0.00pp; grid_path heads-
flat +0.23pp; ARC OOD heads-single +5.80pp top-1 and -1.2020 NLL, with only 20/72 plateau
rows and 0/12 domain/variant verdicts audit-ready.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C7.tiny_net_plateau_stop_rule",
    status: "underpowered",
    severity: "scope",
    title: "Tiny-net architecture claims are gated by plateau audits",
     text: "tiny-net plateau audit covers 6 capacities, 3 restarts, and 8 checkpoints
through 128 epochs across 12 domain/variant verdicts; 20/72 required rows meet the
plateau threshold and 0/12 verdicts are audit-ready.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C8.trace_determinism_audit",
    status: "contradicted",
    severity: "scope",
    title: "Trace determinism audit catches replay-sensitive claim drift",
   text: "the trace audit checks whether replay meaning survives numeric/runtime changes:
```

```
2 traces and 2000 rows, 0 invalid rows, 0 rank-sensitive rows, 0 logged mismatches, same
semantic hash: no.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C5.arc_verified_self_play",
    status: "underpowered",
    severity: "scope",
    title: "ARC verified self-play is a fixed-budget search-efficiency claim",
      text: "ARC verified self-play is registered but underpowered: missing build/
float/arc_verified_self_play/summary.jsonl, so no public replay, fixed-budget ablation,
leakage audit, or zero-family opening can be credited.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C9.signal_mining_replay_options",
    status: "underpowered",
    severity: "scope",
     title: "Signal mining option-ranker evaluates bounded policy hypotheses through
replay",
      text: "Signal mining claim registered as underpowered pending summary artifact
generation.",
    roadmap_items: (),
    residual: (),
  ),
)
```

The promoted spine covers three task families with exact-check and replay verifiers. The sandbox tasks (ARC, program synthesis, haiku, signal) are experimental proving grounds. Extending coverage requires broader verifiers: visual-algebra programs for ARC, DSL correctness checkers for program synthesis, and semantic understanding transfers for haiku.

The holographic memory substrate is demonstrated at d up to 2048 with hundreds of stored rules. Scaling to millions of rules requires dimension scaling (crosstalk scales as 1/sqrt(d)) or hierarchical VSA structures. The EML binding operation's degradation of memory recall (1.4% vs 48.6%) means EML neurons should be confined to the representation-learning layers, not the memory storage path.

The EML scorer improvements are validated on synthetic benchmarks and the three promoted experiments. The universal-primitive property is demonstrated through stacked layer composition but not yet through formal function-approximation bounds for the specific architecture. The pairwise layer requires even layer widths and pre-activation clipping; these constraints are manageable for the compact networks used here but may limit applicability to very deep architectures.

The dataset-first engine currently enforces a fixed 300-epoch, hidden-width-24 baseline. While this provides a consistent calibration point, it does not represent the optimal configuration for any individual task.

13. Conclusion

We presented an integrated architecture for verifier-grounded neural search. Three components compose: a certified gate framework that separates validity from preference through machine-checked symbolic rules, a holographic memory substrate that stores rules as vector bindings with 1/sqrt(N)

retrieval fidelity on the unit hypersphere, and an EML-augmented neural scorer that improves ranking accuracy through a universal continuous primitive.

The integration is honest about what each component contributes and where it fails. EML does not improve binary-feature domains (prime_gap, grid_path). Holographic memory recall degrades under EML binding (1.4% vs 48.6%). Certified gates do not claim to operate outside their declared verifier support. But together they form a disciplined pipeline: enumerate, check, rank, trace, claim—with every assertion bound to machine-readable trace artifacts and mechanically verified by the claim compiler.

The dataset-first engine turns the pipeline into a repeatable process: any experiment that exports candidate rows and verifier labels under the shared schema can enter the evidence matrix. The holographic store preserves what is learned as retrievable context. The EML scorer provides ranking pressure where continuous structure exists. The claim compiler ensures that the paper never says more than the evidence supports.

Bibliography

[1] R. Alur and others, "Automata-Based Verification for Neural Systems," 2024.

[2] L. Gao and others, "Program Synthesis with Large Language Models," 2024.

[3] S. Polu and I. Sutskever, "Generative Language Modeling for Automated Theorem Proving," *arXiv preprint arXiv:2009.03393*, 2020.

[4] D. Kleyko *et al.*, "Vector Symbolic Architectures as a Computing Framework for Nanoscale Hardware and Emerging Applications," *Proceedings of the IEEE*, vol. 110, no. 10, pp. 1538–1571, 2022, doi: 10.1109/JPROC.2022.3209104.

[5] T. A. Plate, *Holographic Reduced Representations*. CSLI Publications, 2003.

[6] rati, "leOS-C – Holographic Engine in C." [Online]. Available: https://github.com/ratimics/leos-c

[7] Anonymous, "The EML Operator: A Universal Primitive for Continuous Mathematics." 2026.